APOSTILA

PROGRAMAÇÃO ORIENTADA A OBJETOS EM JAVA

Conceitos, Princípios e Prática
para Desenvolver Soluções Modernas



Marcos Monteiro Junior
PROFESSOR

Programação Orientada a Objetos em Java

Marcos Monteiro Junior

Conceitos Princípios e Prática

Sumário

1 Organização do projeto, tratamento de exceções e conexão com banco de dados **4**

1.1 JDBC *Java Database Connectivity* 4

1.1.1 JDBC 4

1.1.2 Correspondência entre tipos de dados SQL e Java 8

1.2 Instruções SQL em Java 8

1.2.1 `Statement` 8

1.2.2 `PreparedStatement` 11

1.3 Fábrica de conexões 15

1.4 Conclusão 16

Referências Bibliográficas **17**

1 Organização do projeto, tratamento de exceções e conexão com banco de dados

1.1 JDBC *Java Database Connectivity*

No desenvolvimento de software é muito comum a mudança de tecnologia e uma delas é a mudança de banco de dados. Cada troca causa transtornos para equipe de desenvolvimento. Com o passar dos anos, novas tecnologias facilitaram e tornaram essa mudança (específicamente de banco de dados) menos dolorosa. A aruitetura MVC foi uma delas, desacoplar a estrutura de código do banco de dados do restante do sistema é o primeiro passo para facilitar a manutenção e alterações mais profundas.

Uma outra tecnologia foi o desenvolvimento e ferramentas que realizam uma espécie de "interface" entre a liguagem de programação e o SQL de banco de dados, tornando o desenvolvimentos de persistência uma tarefa única e mais simples.

Observação

O termo persistência em desenvolvimento de software se refere à capacidade de armazenar dados de forma duradoura, permitindo que eles sejam recuperados e utilizados mesmo após o encerramento do programa ou do sistema. Em outras palavras, a persistência garante que as informações não sejam perdidas quando a aplicação é finalizada, sendo armazenadas em um meio de armazenamento permanente, como bancos de dados, arquivos ou sistemas de gerenciamento de dados.

1.1.1 JDBC

O JDBC foi criado em conjunto com os grandes lideres de mercado de banco de dados, como Oracle, IBM e outras. O objetivo era criara uma interface padrão para que qualquer banco de dados pudesse ser acessado por qualquer aplicação Java. O JDBC é uma API que permite a comunicação de diferentes banco de dados ao Java. Ela se fundamenta em três pilares:

- O JDBC deve ser uma API de nível SQL.
- O JDBC deve aproveitar a experiência de APIs de banco de dados existentes.
- O JDBC deve ser simples.

O desenvolvedor não deixará de utilizar o SQL. Mas o JDBC permite que você faça a transição suave entre o banco de dados e da aplicação Java. Seus resultados do banco de dados, por exemplo, são retornados como objetos Java, e os problemas de acesso são lançados como exceções Reese (2000).

A arquitetura do JDBC é apresentada na figura abaixo:

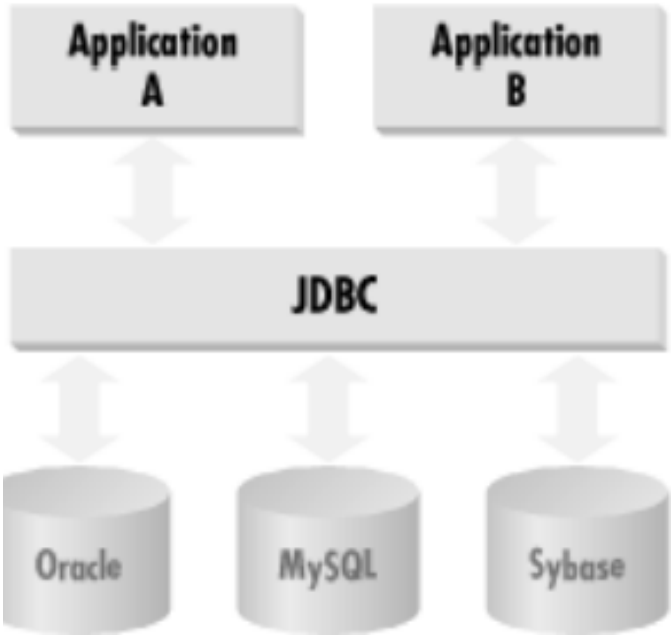


Figura 1.1: Arquitetura do JDBC
Fonte: Reese (2000)

O JDBC possui uma estrutura de classes que permitem a conexão com o banco de dados e tratamento de strings para geração de consultas SQL. A figura abaixo apresenta a estrutura de classes do JDBC.

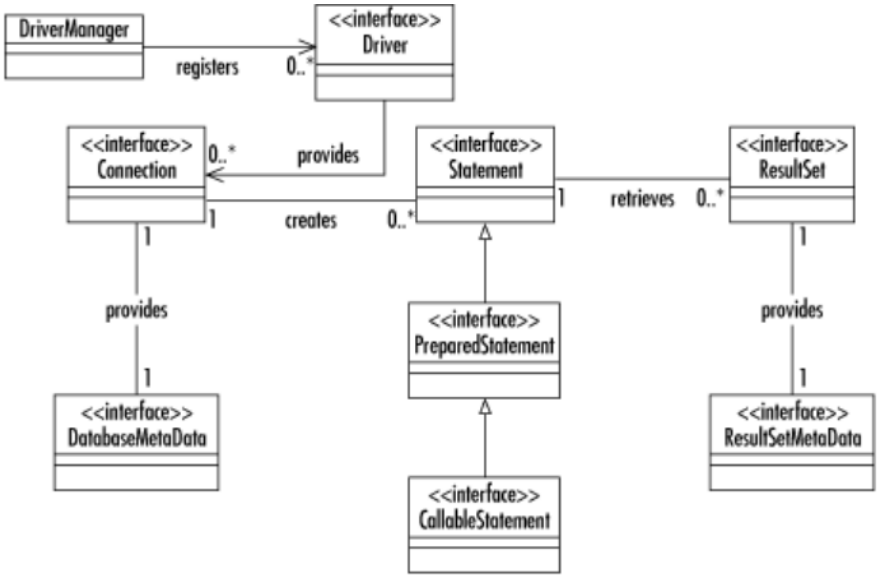


Figura 1.2: Estrutura de classes do JDBC
Fonte: Reese (2000)

Uma das classes mais importantes do JDBC é a classe `DriverManager`, que gerencia os drivers de banco de dados e estabelece conexões com eles. Para se conectar a um banco de dados, você precisa fornecer a URL do banco, o nome de usuário e a senha. Além disso, você deve carregar o driver especifico do banco de dados que deseja utilzar.

Dica

1. Baixe o driver JDBC do MySQL no site oficial: `https://dev.mysql.com/downloads/connector/j/`.
2. Escolha a versão compatível com o seu sistema operacional e clique em "Download".
3. Extraia o arquivo .zip baixado e localize o arquivo mysql-connector-java-X.X.XX.jar.
4. No IntelliJ, vá em File > Project Structure (ou pressione Ctrl + Alt + Shift + S).
5. No menu lateral esquerdo, selecione Modules.
6. Clique na aba Dependencies na parte superior.
7. Clique no ícone de + e escolha JARs or directories....
8. Selecione o arquivo .jar do MySQL que você extraiu e clique em OK

`java.sql.Driver`

Utilizada somente para carregar o driver do banco de dados. Para manipular essa classe, você deve utilziar os métodos dessa clase se você precisa de algo espcíco para sua aplicação.

`java.sql.Connection`

Utilizada para estabelecer uma conexão com o banco de dados. Ela permite a execução de consultas SQL e o controle de transações.

`java.sql.Statement`

Utilizada para executar instruções SQL simples. É a classe mais básica para realizar operações no banco de dados.

`java.sql.PreparedStatement`

Utilizada para executar instruções SQL pré-compiladas. Ela oferece melhor desempenho e segurança em relação à classe `Statement`.

`java.sql.ResultSet`

Utilizada para armazenar e manipular os resultados de uma consulta SQL. Ela permite a navegação pelos dados retornados pelo banco de dados.

Um exemplo de conexão com o banco de dados utilizando JDBC é apresentado abaixo:

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  public class ConexaoBanco {
4      public static void main(String[] args) {
5          String url = "jdbc:mysql://localhost:3306/meubanco";
6          String usuario = "root";
7          String senha = "senha";
8
9          try {
10             // Carrega o driver do MySQL
11             Class.forName("com.mysql.cj.jdbc.Driver");
12
13             // Estabelece a conexão com o banco de dados
14             Connection conexao = DriverManager.getConnection(url, usuario
       , senha);
15             System.out.println("Conexão estabelecida com sucesso!");
16
17             // Fechar a conexão após o uso
18             conexao.close();
19         } catch (Exception e) {
20             System.out.println("Erro ao conectar ao banco de dados: " + e
       .getMessage());
21         }
22     }
```

```
}
```

1.1.2 Correspondência entre tipos de dados SQL e Java

Para trabalhar com banco de dados, é importante entender como os tipos de dados SQL correspondem aos tipos de dados em Java. A tabela 1.1 abaixo apresenta uma correspondência comum entre os tipos de dados SQL e Java:

Tabela 1.1: Correspondência entre tipos de dados SQL e Java

Tipo SQL	Tipo Java
INT	int
VARCHAR	String
DATE	Date
BOOLEAN	boolean

Para outros tipos é importante consultar a documentação do banco de dados e do JDBC para garantir a correspondência correta entre os tipos de dados.

1.2 Instruções SQL em Java

Nessa seção serão apresentadas as instruções SQL mais comuns utilizadas em Java para manipulação de dados em um banco de dados. As instruções SQL são executadas através das classes `Statement` e `PreparedStatement` do JDBC.

1.2.1 `Statement`

A classe `Statement` é utilizada para executar instruções SQL simples, como consultas, inserções, atualizações e exclusões. A seguir, é demonstrado exemplos de como utilizar a classe `Statement` para realizar operações básicas em um banco de dados.

Inserção de dados

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class ExemploMySQLInsert {
    public static void main(String[] args) {
        // URL de conexão do MySQL (substitua 'sua_base', 'seu_usuario' e 'sua_senha')
        String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&serverTimezone=UTC";
        String usuario = "seu_usuario";
        String senha = "sua_senha";

```

```
12          try (Connection conexao = DriverManager.getConnection(url,
    usuario, senha);
13               Statement statement = conexao.createStatement()) {
14
15              String sql = "INSERT INTO clientes (nome, email) VALUES ('
    Maria Souza', 'maria@email.com')";
16
17              // Executa o comando de modificação no banco
18              int linhasAfetadas = statement.executeUpdate(sql);
19
20              System.out.println("Registro inserido no MySQL com sucesso!
    Linhas afetadas: " + linhasAfetadas);
21
22          } catch (Exception e) {
23              System.out.println("Erro ao executar o Statement no MySQL: "
    + e.getMessage());
24          }
25      }
26  }
```

Consulta de dados

```
1   import java.sql.Connection;
2   import java.sql.DriverManager;
3   import java.sql.ResultSet;
4   import java.sql.Statement;
5
6   public class ExemploMySQLSelect {
7       public static void main(String[] args) {
8           String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&
    serverTimezone=UTC";
9           String usuario = "seu_usuario";
10          String senha = "sua_senha";
11
12          try (Connection conexao = DriverManager.getConnection(url,
    usuario, senha);
13               Statement statement = conexao.createStatement()) {
14
15              String sql = "SELECT id, nome, email FROM clientes";
16
17              // Executa a consulta
18              ResultSet resultado = statement.executeQuery(sql);
19
20              // Percorre os registros retornados
21              while (resultado.next()) {
22                  int id = resultado.getInt("id");
23                  String nome = resultado.getString("nome");
24                  String email = resultado.getString("email");
25
26                  System.out.println("ID: " + id + " | Nome: " + nome + " |
     E-mail: " + email);
27              }
```

```
28
29          } catch (Exception e) {
30              System.out.println("Erro ao consultar dados no MySQL: " + e.
    getMessage());
31          }
32      }
33  }
```

Atualização de dados

```
1       import java.sql.Connection;
2   import java.sql.DriverManager;
3   import java.sql.Statement;
4
5   public class ExemploMySQLUpdate {
6       public static void main(String[] args) {
7           // Configurações de conexão com o MySQL
8           String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&
    serverTimezone=UTC";
9           String usuario = "seu_usuario";
10          String senha = "sua_senha";
11
12          // O try-with-resources garante que a conexão e o statement serão
     fechados automaticamente
13          try (Connection conexao = DriverManager.getConnection(url,
    usuario, senha);
14               Statement statement = conexao.createStatement()) {
15
16              // Instrução SQL para atualizar o registro onde o ID é igual
    a 1
17              String sql = "UPDATE clientes SET email = '
    novo_email_carlos@email.com' WHERE id = 1";
18
19              // O método executeUpdate é usado para INSERT, UPDATE e
    DELETE.
20              // Ele retorna um inteiro indicando quantas linhas foram
    modificadas no banco.
21              int linhasAfetadas = statement.executeUpdate(sql);
22
23              if (linhasAfetadas > 0) {
24                  System.out.println("Registro atualizado com sucesso!
    Linhas modificadas: " + linhasAfetadas);
25              } else {
26                  System.out.println("Nenhum registro foi encontrado com o
    ID especificado para atualização.");
27              }
28
29          } catch (Exception e) {
30              System.out.println("Erro ao executar o comando de atualização
    : " + e.getMessage());
31          }
32      }
```

```
}
```

Exclusão de dados

```
    import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class ExemploMySQLDelete {
    public static void main(String[] args) {
        // Configurações de conexão com o MySQL
        String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&serverTimezone=UTC";
        String usuario = "seu_usuario";
        String senha = "sua_senha";

        // O try-with-resources garante o fechamento automático da conexão e do statement
        try (Connection conexao = DriverManager.getConnection(url, usuario, senha);
             Statement statement = conexao.createStatement()) {

            // Instrução SQL para excluir o cliente que possui o ID igual a 1
            String sql = "DELETE FROM clientes WHERE id = 1";

            // Executa a exclusão. O executeUpdate retorna o número de linhas removidas.
            int linhasAfetadas = statement.executeUpdate(sql);

            if (linhasAfetadas > 0) {
                System.out.println("Registro excluído com sucesso! Linhas removidas: " + linhasAfetadas);
            } else {
                System.out.println("Nenhum registro foi encontrado com o ID especificado para exclusão.");
            }

        } catch (Exception e) {
            System.out.println("Erro ao executar o comando de exclusão: " + e.getMessage());
        }
    }
}
```

1.2.2 PreparedStatement

Esse conceito é fundamental no contexto de programação de bancos de dados com JDBC, especialmente quando estudamos a diferença entre as interfaces Statement e PreparedStatement. Enquanto um Statement comum passa por todo o processo de análise (parsing) e

construção de um novo plano de consulta a cada execução, o PreparedStatement permite que o banco de dados reaproveite o plano de consulta, melhorando significativamente a performance em execuções repetidas Reese (2000).

veja o mesmos exemplos utilizando o PreparedStatement:

Inserção de dados

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class ExemploPreparedStatementInsert {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&
    serverTimezone=UTC";
        String usuario = "seu_usuario";
        String senha = "sua_senha";

        // Note o uso de '?' nos locais onde entram os valores dinâmicos
        String sql = "INSERT INTO clientes (nome, email) VALUES (?, ?)";

        try (Connection conexao = DriverManager.getConnection(url,
    usuario, senha);
             PreparedStatement pstmt = conexao.prepareStatement(sql)) {

            // Atribuindo os valores aos parâmetros (indexados a partir
    de 1)
            pstmt.setString(1, "Maria Souza");
            pstmt.setString(2, "maria@email.com");

            // Executa a inserção
            int linhasAfetadas = pstmt.executeUpdate();

            System.out.println("Registro inserido com sucesso via
    PreparedStatement! Linhas afetadas: " + linhasAfetadas);

        } catch (Exception e) {
            System.out.println("Erro ao executar o PreparedStatement de
    inserção: " + e.getMessage());
        }
    }
}
```

Consulta de dados

```
    import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

```

```
public class ExemploPreparedStatementSelect {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&serverTimezone=UTC";
        String usuario = "seu_usuario";
        String senha = "sua_senha";

        // Exemplo filtrando por e-mail (usando parâmetro '?')
        String sql = "SELECT id, nome, email FROM clientes WHERE email = ?";

        try (Connection conexao = DriverManager.getConnection(url, usuario, senha);
             PreparedStatement pstmt = conexao.prepareStatement(sql)) {

            // Define o valor do parâmetro da consulta
            pstmt.setString(1, "maria@email.com");

            try (ResultSet resultado = pstmt.executeQuery()) {
                // Percorre os registros retornados
                while (resultado.next()) {
                    int id = resultado.getInt("id");
                    String nome = resultado.getString("nome");
                    String email = resultado.getString("email");

                    System.out.println("ID: " + id + " | Nome: " + nome + " | E-mail: " + email);
                }
            }

        } catch (Exception e) {
            System.out.println("Erro ao consultar dados com PreparedStatement: " + e.getMessage());
        }
    }
}
```

Atualização de dados

```
    import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;

public class ExemploPreparedStatementUpdate {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&serverTimezone=UTC";
        String usuario = "seu_usuario";
        String senha = "sua_senha";

        // Atualiza o e-mail filtrando pelo ID do cliente
        String sql = "UPDATE clientes SET email = ? WHERE id = ?";
```

```
13
14          try (Connection conexao = DriverManager.getConnection(url,
    usuario, senha);
15               PreparedStatement pstmt = conexao.prepareStatement(sql)) {
16
17              // Atribuindo os valores aos '?' na ordem em que aparecem
18              pstmt.setString(1, "novo_email_maria@email.com"); // Primeiro
     '?'
19              pstmt.setInt(2, 1);                                 // Segundo
    '?' (ID)
20
21              int linhasAfetadas = pstmt.executeUpdate();
22
23              if (linhasAfetadas > 0) {
24                  System.out.println("Registro atualizado com sucesso via
    PreparedStatement! Linhas modificadas: " + linhasAfetadas);
25              } else {
26                  System.out.println("Nenhum registro encontrado para
    atualizar.");
27              }
28
29          } catch (Exception e) {
30              System.out.println("Erro ao executar a atualização: " + e.
    getMessage());
31          }
32      }
33  }
```

Exclusão de dados

```
1   import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.PreparedStatement;
4
5  public class ExemploPreparedStatementDelete {
6      public static void main(String[] args) {
7          String url = "jdbc:mysql://localhost:3306/sua_base?useSSL=false&
    serverTimezone=UTC";
8          String usuario = "seu_usuario";
9          String senha = "sua_senha";
10
11         // Exclui o registro com base no ID fornecido dinamicamente
12         String sql = "DELETE FROM clientes WHERE id = ?";
13
14         try (Connection conexao = DriverManager.getConnection(url,
    usuario, senha);
15              PreparedStatement pstmt = conexao.prepareStatement(sql)) {
16
17             // Define o ID do cliente que será excluído
18             pstmt.setInt(1, 1);
19
20             int linhasAfetadas = pstmt.executeUpdate();
```

```
21
22              if (linhasAfetadas > 0) {
23                  System.out.println("Registro excluído com sucesso via
    PreparedStatement! Linhas removidas: " + linhasAfetadas);
24              } else {
25                  System.out.println("Nenhum registro encontrado com o ID
    especificado para exclusão.");
26              }
27
28          } catch (Exception e) {
29              System.out.println("Erro ao executar a exclusão: " + e.
    getMessage());
30          }
31      }
32  }
```

1.3 Fábrica de conexões

Nos exemplos anteriores, todas as classes eram responsáveis por criar a conexão com o banco de dados. Isso não é uma boa prática, pois se houver uma mudança na URL, usuário ou senha do banco, será necessário alterar todas as classes que fazem a conexão. Vimos que em programação orientada a objetos que podemos criar uma classe para centralizar essa lógica. Para resolver esse problema, podemos criar uma classe chamada `FabricaConexao` que centraliza a criação da conexão.

O exemplo abaixo mostra como criar uma fábrica de conexões para o banco de dados MySQL:

```
1  import java.sql.Connection;
2  import java.sql.DriverManager;
3
4  public class conexaoFactory {
5      private static final String URL = "jdbc:mysql://localhost:3306/
       sua_base?useSSL=false&serverTimezone=UTC";
6      private static final String USUARIO = "seu_usuario";
7      private static final String SENHA = "sua_senha";
8
9      public static Connection getConnection() throws Exception {
10         return DriverManager.getConnection(URL, USUARIO, SENHA);
11     }
12 }
```

Agora veja o exemplo de como utilizar a fábrica de conexões em uma classe que realiza operações no banco de dados:

```
1  import java.sql.Connection;
2  import java.sql.SQLException;
3
4  public class TesteConexao {
5      public static void main(String[] args) {
6          System.out.println("Tentando conectar ao banco de dados...");
7
```

```
8           // O try-with-resources abre a conexão e a fecha automaticamente
    ao sair do bloco
9           try (Connection conexao = ConnectionFactory.getConnection()) {
10
11              if (conexao != null && !conexao.isClosed()) {
12                  System.out.println("SUCESSO: Conexão com o MySQL
    estabelecida com sucesso!");
13
14                  // Exemplo opcional: exibir metadados da conexão
15                  System.out.println("Nome do Banco: " + conexao.
    getMetaData().getDatabaseProductName());
16                  System.out.println("Versão do Driver: " + conexao.
    getMetaData().getDriverVersion());
17              }
18
19          } catch (SQLException e) {
20              System.out.println("FALHA: Não foi possível conectar ao banco
     de dados.");
21              System.out.println("Erro técnico: " + e.getMessage());
22          }
23      }
24  }
```

1.4 Conclusão

Neste capítulo, exploramos o desenvolvimento de uma aplicação Java completa, abordando desde a concepção até a implementação. Discutimos a importância da arquitetura MVC para organizar o código e facilitar a manutenção, além de detalhar o tratamento de exceções em Java, garantindo que a aplicação lide com erros de forma robusta.

Além disso, apresentamos o JDBC como uma ferramenta essencial para a comunicação entre a aplicação Java e bancos de dados, demonstrando como realizar operações básicas de inserção, consulta, atualização e exclusão de dados. Também destacamos a importância da correspondência entre tipos de dados SQL e Java, bem como a criação de uma fábrica de conexões para centralizar a lógica de conexão com o banco de dados.

Referências Bibliográficas

Reese, G. (2000). *Database Programming with JDBC and Java*. O'Reilly Media, 2 edition.